

Projet : Site Web pour l'Organisation

Résumé complet de la discussion technique et fonctionnelle

1. Contexte du projet

Une femme dirige une organisation de lutte contre les agressions sexuelles et physiques faites aux femmes et aux jeunes filles. L'utilisateur (Vadeck, étudiant en génie logiciel) propose de créer bénévolement un site internet pour cette organisation afin d'accroître sa visibilité, faciliter les dons et offrir un espace sécurisé aux victimes.

2. Structure du site (pages principales)

- Accueil — message d'impact, chiffres clés, appel à l'action
- Qui sommes-nous — histoire de la fondatrice, mission, valeurs
- Nos actions / Programmes — sensibilisation, accompagnement juridique, soutien psychologique
- Ressources d'urgence — numéros, procédure de signalement
- Témoignages — anonymes, avec modération avant publication
- Comment aider — dons, bénévolat, partenariats
- Actualités / Blog
- Partenaires
- Contact

Points de sécurité spécifiques

- Bouton « sortie rapide » (quick exit) redirigeant instantanément vers un site anodin
- Aucune photo ou détail identifiable des victimes sans consentement écrit
- Page de confidentialité claire sur la protection des données

3. Système de témoignages anonymes

Chaque visiteur peut soumettre un témoignage sans créer de compte. Le texte passe par une file de modération avant toute publication publique, afin d'éviter la diffusion d'informations identifiantes ou dangereuses.

- Formulaire simple : contenu + catégorie, aucune identité requise
- Email de contact optionnel (jamais publié, utile pour un suivi)
- Statut du témoignage : en_attente / valide / rejete
- Un membre de l'équipe relit et valide ou rejette avant publication
- Limite de caractères + captcha pour limiter les abus/spam

Suivi du statut sans compte utilisateur

Décision retenue : combiner un code de suivi anonyme généré automatiquement (ex. TEM-8X3K9) avec un email optionnel pour recevoir une notification automatique. La personne peut ainsi vérifier le statut de son témoignage (publié / non publié) sans jamais fournir son identité.

4. Authentification

Aucun système de compte pour les visiteurs : cela casserait l'anonymat recherché pour les témoignages et créerait de la friction. Seule l'équipe de l'organisation dispose d'un compte (table admins), protégé par email/mot de passe et JWT, pour accéder à l'espace de modération et de gestion de contenu.

5. Stack technique retenue

- Frontend : React (React Router, Tailwind CSS ou Material UI, React Hook Form)
- Backend : Node.js avec Express.js
- Base de données : PostgreSQL
- Authentification admin : JWT
- Hébergement envisagé : Vercel/Netlify (frontend), Render/Railway (backend), Supabase ou base PostgreSQL managée

6. Arborescence du projet

Monorepo avec un dossier frontend/ et un dossier backend/ séparés, chacun avec son propre fichier .env (variables différentes et déploiements distincts). Fichiers .env, .env.example et .gitignore à la racine du projet.

```
mon-projet/
├── .env / .env.example / .gitignore / README.md
├── frontend/
│   ├── src/components/ (common, testimonials, admin)
│   ├── src/pages/ (Home, About, Programs, EmergencyResources,
│   │               Testimonials, HowToHelp, News, Contact, admin/)
│   ├── src/services/ (api.js, testimonialsService.js, ...)
│   ├── src/context/, hooks/, utils/, App.jsx, main.jsx
│   └── backend/
│       ├── src/config/db.js
│       ├── src/models/ (testimonial, admin, contactMessage, program, news, partner)
│       ├── src/controllers/ , routes/ , middlewares/ (auth, rateLimiter, errorHandler)
│       ├── src/utils/ (generateTrackingCode.js, sendEmail.js)
│       └── src/app.js , server.js
```

7. Structure de la base de données PostgreSQL

Un script SQL complet a été généré (fichier séparé : database_schema.sql), contenant 8 tables.

admins — Comptes de l'équipe (email, password_hash, role) — seule authentification du site

testimonials — Témoignages anonymes : tracking_code, content, category, status, contact_email optionnel, moderated_by, moderation_note

contact_messages — Messages du formulaire de contact avec statut (non_lu / lu / traite)

programs — Programmes/actions de l'organisation

emergency_resources — Numéros et ressources d'urgence par catégorie

news — Actualités/blog avec statut de publication

partners — ONG et partenaires affichés sur le site

donations — Suivi optionnel des dons (montant, statut de paiement, référence de transaction)

Points clés du schéma

- Aucune colonne de nom/identité dans testimonials : l'anonymat est garanti au niveau structurel
- tracking_code (UNIQUE) permet à l'utilisateur de suivre son témoignage sans compte
- contact_email jamais exposé par l'API publique, réservé à un usage interne
- Contraintes CHECK sur les colonnes de statut pour éviter les valeurs incohérentes
- Trigger automatique remplissant moderated_at lors du passage à valide/rejete
- Index sur status, tracking_code et slug pour les performances

8. Prochaines étapes envisagées

- Créer le script Node.js pour générer le premier compte admin (hash bcrypt)
- Mettre en place la connexion PostgreSQL (config/db.js)
- Écrire les routes Express : soumission de témoignage, vérification du statut par tracking_code, authentification admin, modération
- Construire les pages React (formulaire de témoignage, page de suivi, dashboard admin)

Note : le fichier database_schema.sql contenant le script SQL complet a été généré séparément lors de la conversation et doit être conservé avec ce résumé pour reprendre le projet.